# Computer Architecture Lab

## (EL-2012)

## LABORATORY REPORT

## Fall 2026




# (Lab_06_)

## Use of Venus Simulator for RISC-V Procedure Execution

Student Name: ___Areeb Fatima_____ Roll No: _____24I-6543_____ Section: _B__

Date performed: _____30th September_____, 2026

**MARKS AWARDED: _____ / 10**

**Signature and date of Instructor: _____**
**Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad**

## Task 1:

**Description:**

In this task, we were given the code and we have to analyze and correct the value for t1 to display the correct addition, so basically when the main calls the procedure then the s0 value is overwritten from 2500 to 3 so, when we call that again after procedure ends so it sums up with 3 not 2500. So, for that we store the value of s0 using stack pointer and then load the stored value before the procedure ends. In this way, we got the desired output.

**Code:**

```
.text

main:

li a0, 1

li s0, 2500

jal ra, procedure                    # call procedure

add t1, s0, a0                # result should be 1003+2500

li a0,1                                    # setup system call for display (command in a0)

mv a1, t1                                  # data to display in a1

ecall                                      # displays result on console

procedure:

addi sp, sp, -4

sw s0, 0(sp)

li s0, 3

li t0, 1000

add a0, s0, t0

lw s0, 0(sp)

addi sp, sp, 4

jalr zero, ra, 0              # return
```

**Output:**




## Task 2:

**Description:**

In this task, we were basically given a code and we have to analyze its working so firstly we took the three words in different three registers and then we use stack pointer for making the space in memory for the three registers as one word took 4 bytes then we store the words with their respective spacing of offset and then load the words for their values appearing on screen and then in last restore the space from stack.

**Code:**

```
addi s0, zero, 5 #s0=5

addi t1, zero, 10 #t1=10

addi t0, zero,15 #t0=15

addi sp, sp, -12 #make space for 3 words(12 bytes)

sw   s0, 0(sp) #store s0(5) in [sp]
```

```
sw   t0, 4(sp) #store t0(15) in [sp+4]

sw   t1, 8(sp) #store t1(10) in [sp+8]

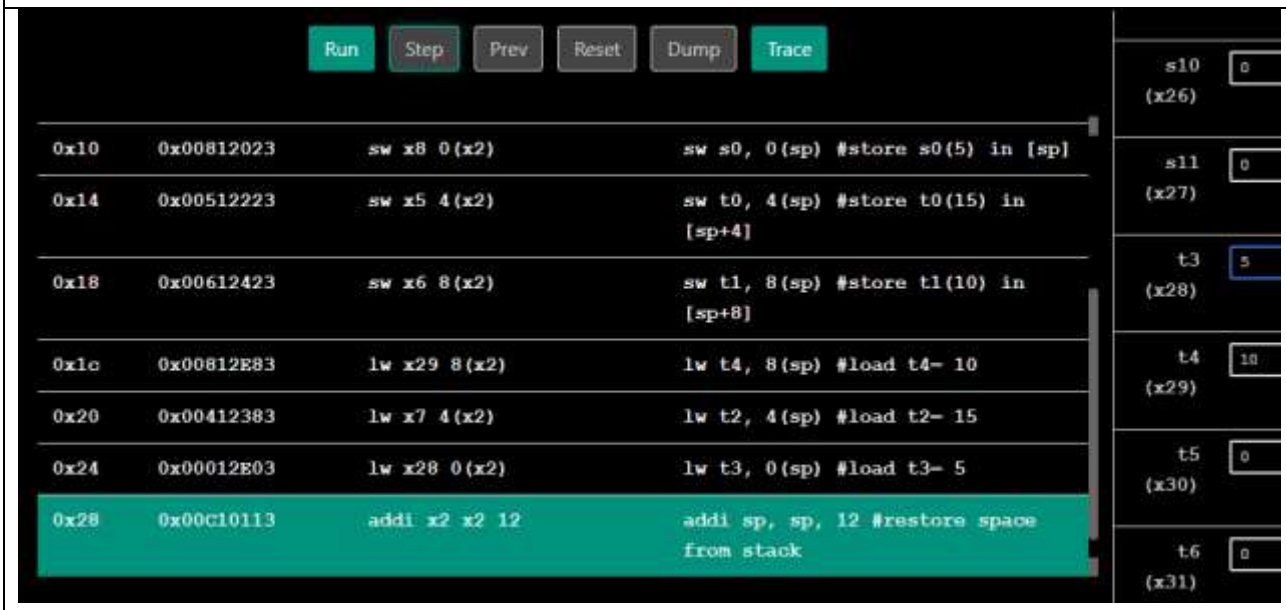
lw   t4, 8(sp) #load t4= 10

lw   t2, 4(sp) #load t2= 15

lw   t3, 0(sp) #load t3= 5

addi sp, sp, 12 #restore space from stack
```

**Output:**




## Task 3:

**Description:**

In this task, we were given a C function and we have to convert it in assembly so we firstly took the words in data and then we load then and then call the procedure and perform the operations of add, sub and mul in it and store the result in a0 and return from procedure then load the word x and store the result in reg a0 and hence we print a0 and ecall to exit the program.

**Code:**

```
.data

a: .word 5
```

```
b: .word 6

x: .word 0

.text

.globl main

la t0, a

lw a0, 0(t0) #a=5

la t1, b

lw a1, 0(t1) #b=6

jal ra, EqSolver

la t5, x

sw a0, 0(t5)

li a7, 1 #print a0

ecall

li a7, 10 #exit

ecall

EqSolver:

add t3, a0, a1

sub t4, a0, a1

mul a0, t3, t4

jalr zero, ra, 0
```

**Output:**




## Task 4:

| **Description:** |
|---|
| In this task, we have to find the area and perimeter as two different procedures and call them in main so, we first load the word and then call jumps to AreaFinder where we mul side by side and then after returning from procedure we store the area in a0 and then print and then we reload the side and call the PerimeterFinder procedure and load 4 and multiply that with side and return from the procedure and store the perimeter in a0 and print that similarly and exit by ecall. |
| **Code:** |

```
.data

side: .word 3

area: .word 0

perimeter: .word 0

.text

.globl main

la t0, side
```

```
lw a0, 0(t0) #side=3

jal ra, AreaFinder

la t1, area

sw a0, 0(t1)

li a7, 1 #print area

ecall

la t0, side #reload side

lw a0, 0(t0) #side=3

jal ra, PerimeterFinder

la t3, perimeter

sw a0, 0(t3)

li a7, 1 #print perimeter

ecall

li a7, 10 #exit

ecall

AreaFinder:

mul a0, a0, a0

jalr zero, ra, 0

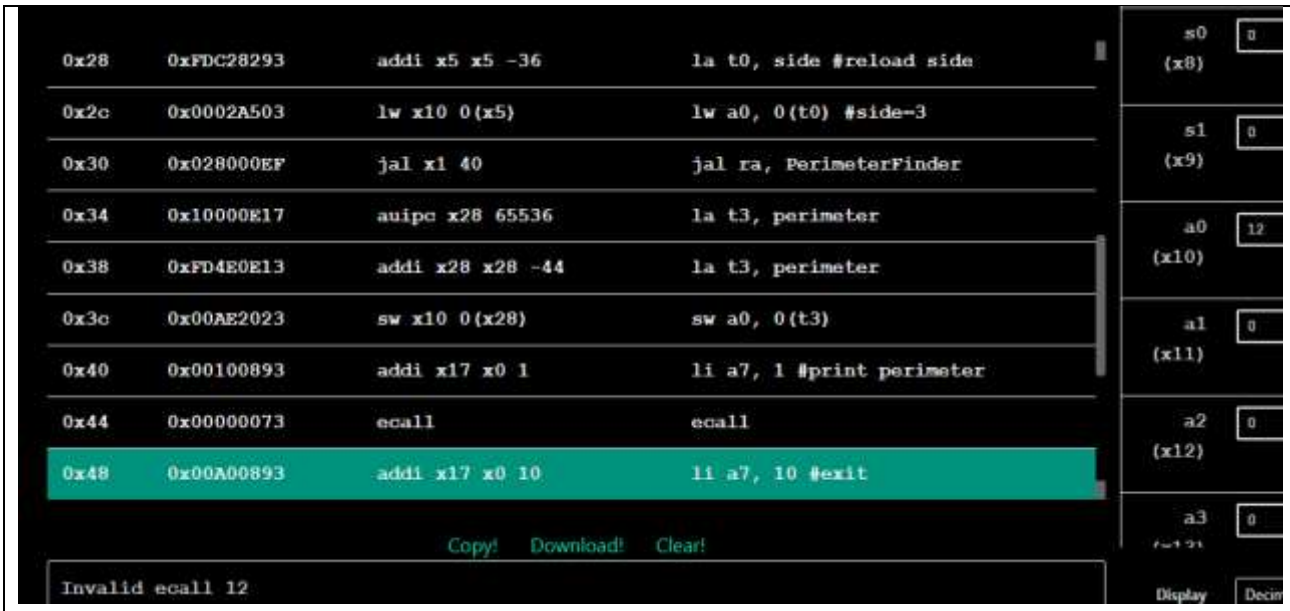
PerimeterFinder:

li t2, 4

mul a0, t2, a0

jalr zero, ra, 0
```

**Output:**




## Task 5:

| **Description:** |
|---|
| In this task, we were given a C function and we have to convert it in assembly so we firstly took the word x in data and then we load then and then jump to the procedure and perform the operations of add and mul and load the integers to find the desired polynomial equation and store that in a0 and return from procedure and then load the word y and store the result in reg a0 and hence we print a0 and ebreak to exit the program. |
| **Code:** |
| .data<br><br>x: .word 2<br><br>y: .word 0<br><br>.text<br><br>.globl main<br><br>la t0, x<br><br>lw a0, 0(t0) #x=2<br><br>jal ra, EqSolver |

```
la t6, y

sw a0, 0(t6)

li a7, 1 #print a0

ecall

li a7, 93 #exit

ecall

ebreak #for exiting

EqSolver:

mul t1, a0, a0 #x^2

mul t2, t1, a0 #x^3

li a1, 5

mul t3, a1, t1 #5x^2

li a2, 6

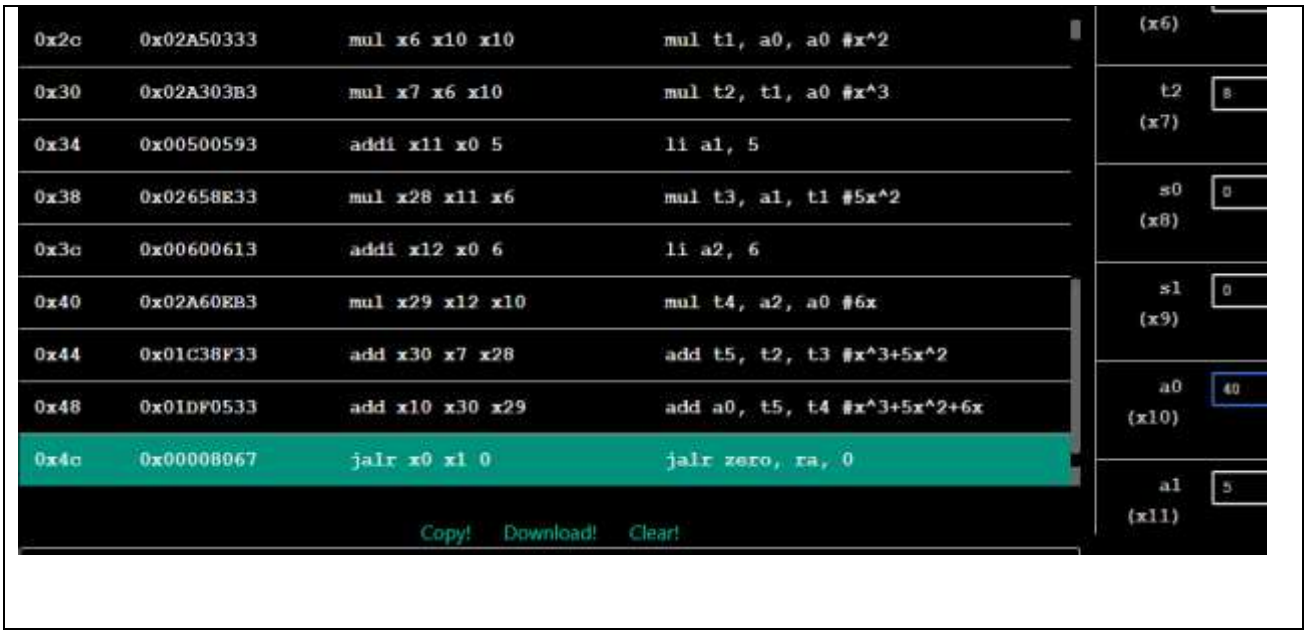
mul t4, a2, a0 #6x

add t5, t2, t3 #x^3+5x^2

add a0, t5, t4 #x^3+5x^2+6x

jalr zero, ra, 0
```

**Output:**




## Task 6:

| **Description:** |
|---|
| In this task, we were given a C program and for writing that in assembly we firstly took the parameters of main in data and then we load those and check the if-else conditions through branch and then if condition fulfilled then call another procedure in which we jump to the procedure and perform arithmetic and return to the procedure and then jump to store result and load the result reg y and store the result in a0 and then print the a0 and ebreak to exit the program. |
| **Code:** |
| .data<br><br>n: .word 2<br><br>k: .word 5<br><br>y: .word 0<br><br>.text<br><br>.globl main<br><br>la t0, n<br><br>lw a0, 0(t0) #a0=n<br><br>la t1, k |

```
lw a1, 0(t1) #a1=k

li a2, 1

beq a1, a2, callF1 #if k==1

bgt a0, a1, callF2 #elseif n>k

j callF3 #else

callF1:

jal ra, F1

j StoreResult

F1:

mul t2, a0, a0 #x^2

mul t3, a1, a1 #z^2

add a0, t2, t3 #x^2+z^2

jalr zero, ra, 0

callF2:

jal ra, F2

j StoreResult

F2:

mul t4, a0, a0 #z^2

mul a0, t4, a0 #z^3

jalr zero, ra, 0

callF3:

jal ra, F3

j StoreResult
```

```
F3:

addi a0, a0, 5 #w+5

jalr zero, ra, 0

StoreResult:

la t4, y

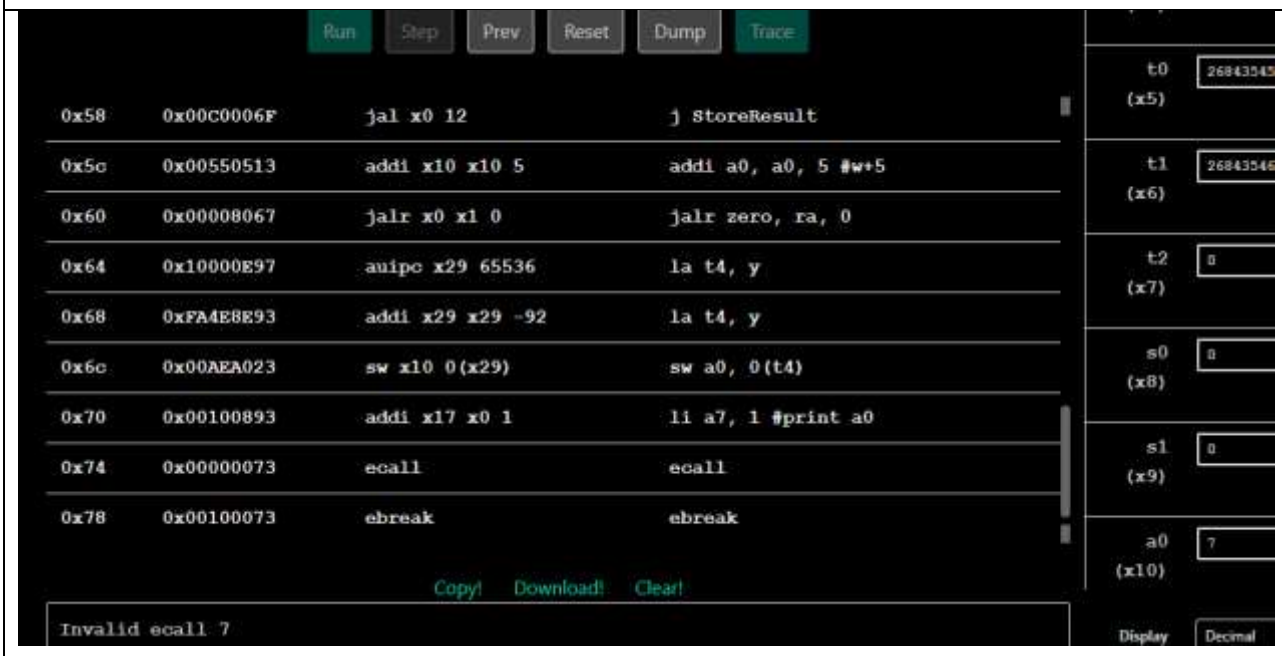
sw a0, 0(t4)

li a7, 1 #print a0

ecall

ebreak
```

**Output:**




## Task 7:

**Description:**

In this task, we were given a C program and for writing that in assembly we firstly took the parameters of main in data and then we load those and check the if-else conditions through branch and then if condition fulfilled then call another procedure in which we jump to the procedure and perform arithmetic and return to the procedure and then jump to store result and load the result reg y and store the result in a0 and then print the a0 and ecall to exit the program.

**Code:**

```
.data

n: .word 5

k: .word 2

y: .word 0

.text

.globl main

la t0, n

lw a0, 0(t0) #a0=n

la t1, k

lw a1, 0(t1) #a1=k

li a2, 0

beq a1, a2, callF1 #if k==0

beq a0, a1, callF2 #elseif n==k

j callF3 #else

callF1:

jal ra, F1

j StoreResult

F1:

mul t2, a0, a0 #x^2

mv a0, t2

jalr zero, ra, 0

callF2:
```

```
jal ra, F2

j StoreResult

F2:

mul t4, a0, a0 #z^2

mul a0, t4, a0 #z^3

jalr zero, ra, 0

callF3:

jal ra, F3

j StoreResult

F3:

addi a0, a0, 1 #w+1

jalr zero, ra, 0

StoreResult:

la t4, y

sw a0, 0(t4)

li a7, 1 #print a0

ecall

ebreak
```

**Output:**




## Task 8:

| **Description:** |
|---|
| In this lab, we basically wrote the assembly code for Fibonacci series with and without recursion. So, first we load the n value then we jump of fib procedure in which we branch for base cases of n=0 and n=1 and then we make space in stack for the recursion and then jump to procedure for finding Fibonacci series. Meanwhile, in without recursion we made the loop by taking a=0 and b=1 and add them in c and then by shifting a to b and b to c we continue this process and return b and then load y and store result in a0. |
| **Code:** |
| **Without Recursion:**<br><br>.data<br><br>n: .word 3<br><br>y: .word 0<br><br>.text<br><br>.globl main<br><br>la t0, n |

```
lw a0, 0(t0) #a0=n

jal ra, fib

la t6, y

sw a0, 0(t6)

li a1, 0

mv a1, a0

li a0, 1 #print a0

ecall

li a0, 10

ecall

fib:

li t1, 0 #a=0

li t2, 1 #b=1

li t3, 2 #i=2

mv t4, t3 #c=i

beq a0, zero, ret_a

li t5, 1

beq a0, t5, ret_b

fib_loop:

add t4, t1, t2 #c=a+b

mv t1, t2 #a=b

mv t2, t4 #b=c

addi t3, t3, 1 #i++
```

```
ble t3, a0, fib_loop

mv a0, t2 #return b

jalr zero, ra, 0

ret_a:

mv a0, t1

jalr zero, ra, 0

ret_b:

mv a0, t2

jalr zero, ra, 0
```

**With Recursion:**

```
.data

n: .word 3

y: .word 0

.text

.globl main

la t0, n

lw a0, 0(t0) #a0=n

jal ra, fib

la t6, y

sw a0, 0(t6)

mv a1, a0

li a0, 1 #print a0

ecall
```

```
li a0, 10 #exit

ecall

fib:

beq a0, zero, ret_a

li t5, 1

beq a0, t5, ret_b

addi sp, sp, -12

sw ra, 8(sp) #save return address

sw a0, 4(sp) #save n

sw t3, 0(sp) #save temporary register

addi a0, a0, -1 #n-1

jal ra, fib #fib(n-1)

mv t3, a0 #store result in t3

lw a0, 4(sp) #restore n

addi a0, a0, -2 #n-2

jal ra, fib #fib(n-2)

add a0, a0, t3 #fib(n-1)+fin(n-2)

lw t3, 0(sp) #restore temp

lw ra, 8(sp) #restore return address

addi sp, sp, 12

jalr zero, ra, 0

ret_a:

li a0, 0
```

```
jalr zero, ra, 0

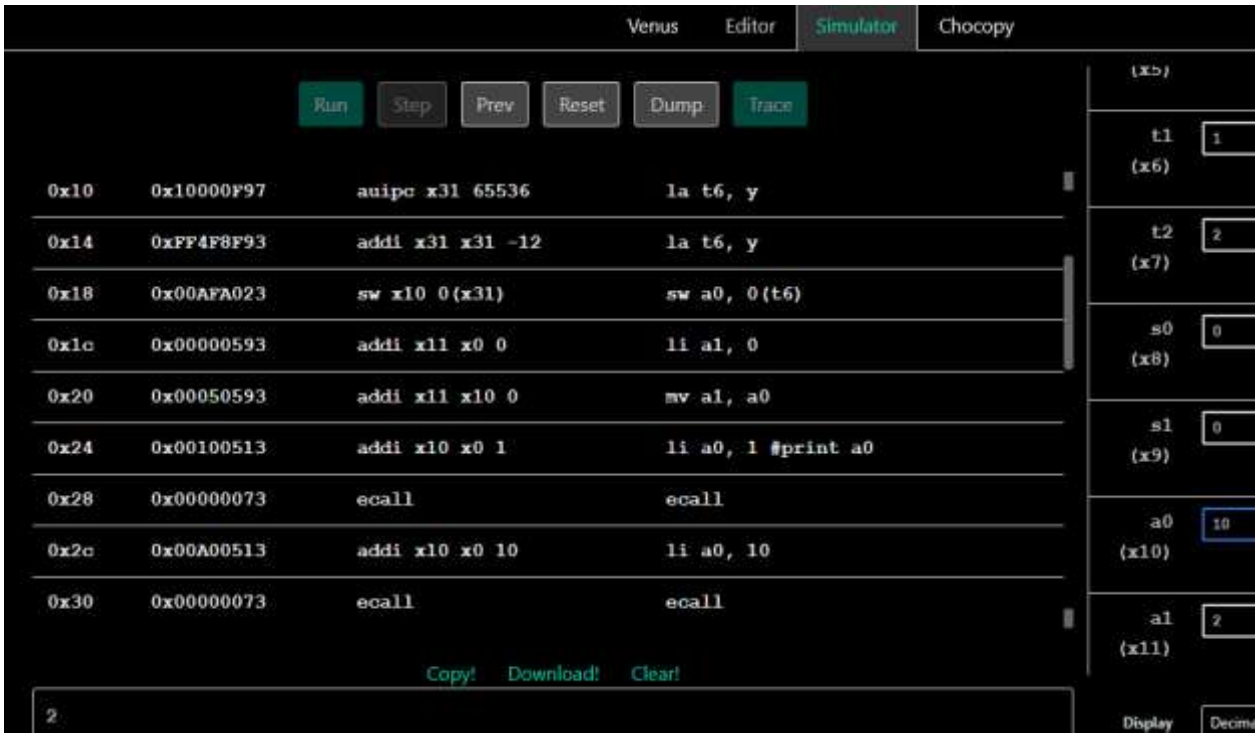
ret_b:

li a0, 1

jalr zero, ra, 0
```

**Output:**

**Without Recursion:**




**With Recursion:**


Venus
Editor
Simulator
Chocopy
Run
Step
Prev
Reset
Dump
Trace
0xc 0x024000EF jal x1 36 jal ra, fib
0x10 0x10000F97 auipc x31 65536 la t6, y
0x14 0xFF4FBF93 addi x31 x31 -12 la t6, y
0x18 0x00AFA023 sw x10 0(x31) sw a0, 0(t6)
0x1c 0x00050593 addi x11 x10 0 mv a1, a0
0x20 0x00100513 addi x10 x0 1 li a0, 1 #print a0
0x24 0x00000073 ecall ecall
0x28 0x00A00513 addi x10 x0 10 li a0, 10 #exit
0x2c 0x00000073 ecall ecall
Copy!
Download!
Clear!
2
t1 (x6) 0
t2 (x7) 0
s0 (x8) 0
s1 (x9) 0
a0 (x10) 10
a1 (x11) 2
Display